

Paragliding Intelligence Platform

Volume 08 - Infrastructure & DevOps v2 Technical

Deployment, operations, reliability, CI/CD, observability, backup, cost and environment architecture

Document status: replacement-quality technical specification. This volume supersedes any earlier placeholder infrastructure notes. It is designed as an implementation blueprint for the next-edge paragliding meteo platform described in Volumes 01-07.

Scope: production platform for a Windy/Ventusky-class weather map, paragliding decision engine, mobile tracking, paid tiers, weather ingestion, AI/ML inference, digital twins, community reports, clubs, schools, events and rescue operations.

Item	Value
Volume	08 - Infrastructure & DevOps
Version	v2 Technical
Primary readers	CTO, DevOps engineer, backend lead, security engineer, mobile lead, product owner
Assumed stack	FastAPI/Python, PostgreSQL/PostGIS, TimescaleDB, Redis, object storage, Kubernetes/K3s, Next.js, Flutter, Celery/worker queues
Design posture	Start affordable on Hetzner; keep cloud-neutral interfaces; allow later AWS/GCP expansion for global scale.

0. Table of Contents

1. Executive Infrastructure Summary
2. Environment Strategy
3. Target Deployment Topology
4. Kubernetes and Runtime Architecture
5. Network and Edge Architecture
6. Storage and State Architecture
7. Weather Pipeline Operations
8. AI/ML Operations
9. CI/CD and Release Management
10. Secrets and Configuration
11. Observability
12. Backup and Disaster Recovery
13. Security Operations Handoff
14. Performance and Scaling
15. Cost Control
16. Runbooks
17. MVP Infrastructure Slice
18. Roadmap and Backlog

1. Executive Infrastructure Summary

The infrastructure must support two very different workload classes: low-latency user-facing APIs and heavy offline weather/ML processing. The user-facing path powers map interactions, flyability decisions, live tracking, alerts and billing. The processing path ingests weather models, radar, station observations, pilot reports, airspace updates, DEM tiles and ML features. These workloads must be isolated so a large weather import cannot degrade launch safety alerts or live tracking.

The recommended foundation is a staged architecture: Docker Compose for local development, a small K3s/Kubernetes cluster for production MVP, and a scale-out Kubernetes topology for V1/V2. Kubernetes is selected because it provides declarative workload management, service discovery, rolling updates, self-healing primitives and a standard abstraction for containerized services. For the early European MVP, Hetzner is a cost-efficient deployment target; the design keeps cloud-specific dependencies behind adapters so that object storage, DNS, load balancers and managed databases can be migrated later.

Infrastructure principle	Technical implication
Separate real-time from batch	tracking-api, alert-engine and public-api run in dedicated node pools; weather ingestion workers can be throttled.
GIS-first storage	PostgreSQL/PostGIS is a first-class core dependency; all launches, airspace, terrain metadata and route objects use spatial indexes.
Time-series native	TimescaleDB hypertables store forecasts, observations, telemetry and forecast error samples.
Object storage for big data	GRIB, NetCDF, Zarr, DEM, radar frames, model artifacts and map tiles are stored in S3-compatible object storage.
Safety over freshness	If a model import is incomplete or inconsistent, the platform must mark data stale instead of generating confident recommendations.
Paid-tier isolation	Premium AI/ML and high-resolution weather endpoints must be rate-limited and metered independently from free map browsing.

2. Environment Strategy

The platform uses four long-lived environments plus ephemeral preview environments. Every environment is deployed from the same Helm charts or Compose descriptors, with different configuration values and secrets. No environment may use hand-edited containers as the source of truth.

Environment	Purpose	Data policy	Deployment
local	Developer workstations and local integration tests	Synthetic seed data only; no user data	Docker Compose with Postgres/PostGIS, Redis, MinIO and local services
dev	Shared integration environment	Synthetic plus small non-sensitive public weather samples	K3s or single-node Kubernetes
staging	Production-like validation	Anonymized production-like data; sanitized telemetry	Kubernetes; same ingress, TLS and observability as prod
production	Public platform	Real users, billing, location history, alerts	HA Kubernetes cluster with backups, monitoring and change control
preview/*	Pull-request validation	Ephemeral synthetic fixtures	Namespace-per-PR or app-per-PR with TTL cleanup

2.1 Naming Conventions

Namespaces:

pip-dev, pip-staging, pip-prod

Services:

public-api, map-api, weather-ingestion, forecast-processor, risk-engine,
ai-briefing, tracking-api, alert-engine, billing-service, admin-api

Databases:

pip_core, pip_weather, pip_tracking, pip_ml

Object buckets:

pip-raw-weather, pip-processed-weather, pip-map-tiles, pip-model-artifacts,
pip-mobile-offline-packs, pip-igc-uploads

2.2 Promotion Rules

1. A commit can reach dev after unit tests, static analysis, container build and schema lint pass.
2. A release candidate can reach staging after integration tests and migration dry-run pass.
3. A production release requires green smoke tests, successful backup check, release notes and rollback command documented.
4. Weather-engine releases must replay a fixed historical model-run sample and compare derived values against known baselines.
5. AI/ML releases must pass model-card gates, offline metrics gates and LLM safety regression tests.

3. Target Deployment Topology

The production MVP can run on a compact cluster, but the logical topology must be production-grade from the beginning. The platform should avoid a single giant monolith because weather workers, maps, tracking, AI and billing scale differently. The first production deployment may use fewer physical nodes while preserving service boundaries.

```
Internet
-> CDN / WAF / DNS
-> Load Balancer
-> Ingress Controller
-> API Gateway
    -> public-api
    -> map-api
    -> weather-api
    -> flyability-api
    -> tracking-api
    -> ai-briefing-api
    -> billing-service
-> Worker queues
    -> weather-ingestion-workers
    -> forecast-processing-workers
    -> tile-generation-workers
    -> alert-workers
    -> ml-inference-workers
-> Data platform
    -> PostgreSQL + PostGIS
    -> TimescaleDB
    -> Redis
    -> S3-compatible object storage
    -> Vector tile cache
```

3.1 Node Pools

Node pool	Workloads	Scaling signal	Notes
edge	ingress, API gateway, public APIs	HTTP latency, CPU, request rate	Protect with rate limits and WAF rules.
compute	forecast processing, tile generation, batch jobs	queue length, CPU, memory	Can use cheaper CPU-heavy instances.
realtime	tracking-api, alert-engine, websocket gateway	WebSocket count, latency, memory	Separate from batch jobs to protect in-flight safety UX.
ml	ML inference, AI context builder, model scoring	GPU/CPU utilization, queue latency	Start CPU-only; add GPU only if CV or LLM self-hosting requires it.
data	database and storage if self-hosted	IOPS, memory, disk	Prefer managed or isolated nodes with strict backup policy.

4. Kubernetes and Runtime Architecture

Kubernetes is the standard target runtime for production and staging. For early deployment, K3s is acceptable if operational simplicity is more important than a managed control plane. The design must remain compatible with standard Kubernetes

objects: Deployments, StatefulSets, Services, Ingress, CronJobs, Jobs, ConfigMaps, Secrets, NetworkPolicies, PodDisruptionBudgets and HorizontalPodAutoscalers.

4.1 Workload Types

Workload	Kubernetes object	Reason
public-api, map-api, weather-api	Deployment	Stateless HTTP services with rolling updates.
tracking-websocket-gateway	Deployment	Horizontally scalable realtime gateway; session state stored externally.
postgres/timescale when self-hosted	StatefulSet	Stable network identity and persistent volumes.
redis	StatefulSet or managed service	Cache and queue broker; persistence mode depends on role.
weather import schedule	CronJob	Runs model-run discovery and import orchestration.
forecast processing batches	Job or queue worker Deployment	Can be retried and scaled by queue depth.
tile generation	Job/worker	CPU intensive, cacheable artifacts.
database migrations	Job	One-shot gated release step.

4.2 Example Kubernetes Manifest

```
apiVersion: apps/v1
kind: Deployment
metadata:
  name: flyability-api
  namespace: pip-prod
spec:
  replicas: 3
  selector:
    matchLabels: { app: flyability-api }
  template:
    metadata:
      labels: { app: flyability-api, tier: api }
    spec:
      containers:
        - name: app
          image: registry.example.com/pip/flyability-api:1.0.0
          ports: [{ containerPort: 8080 }]
          envFrom:
            - configMapRef: { name: pip-runtime-config }
            - secretRef: { name: pip-runtime-secrets }
          readinessProbe:
            httpGet: { path: /health/ready, port: 8080 }
            initialDelaySeconds: 5
            periodSeconds: 10
          livenessProbe:
            httpGet: { path: /health/live, port: 8080 }
            initialDelaySeconds: 20
            periodSeconds: 20
          resources:
            requests: { cpu: "250m", memory: "512Mi" }
            limits: { cpu: "1000m", memory: "1Gi" }
```

4.3 Health Endpoint Contract

Endpoint	Must check	Must not check
/health/live	Process can serve basic loop and has not deadlocked	External database freshness; avoid killing pods during DB outage.
/health/ready	DB connection, Redis connection if mandatory, required config loaded	Long weather import status.
/health/startup	Migration compatibility, warm cache loaded if required	External API calls to forecast providers.
/metrics	Prometheus metrics endpoint	Business-sensitive raw data.

5. Network and Edge Architecture

The edge layer must provide TLS, routing, rate limits, request size limits, bot mitigation, abuse detection and low-latency delivery of static assets. Weather map tiles and mobile offline packs should be served through CDN/object storage paths rather than through the main API whenever possible.

5.1 Edge Flow

User App
-> DNS
-> CDN / WAF
-> Load Balancer
-> Ingress Controller
-> API Gateway
-> Internal service

Map tile path:
User App -> CDN -> object storage / tile-api fallback

WebSocket path:
User App -> LB -> ingress websocket route -> tracking-gateway

5.2 Domain Plan

Hostname	Purpose	Caching
app.example.com	Next.js web app	CDN cache for static assets.
api.example.com	REST API gateway	No cache for authenticated responses unless explicit.
tiles.example.com	Vector tiles, raster overlays, weather frames	Aggressive cache with versioned paths.
ws.example.com	Live tracking and alert WebSockets	No CDN cache; sticky not required if state externalized.
admin.example.com	Internal/admin portal	IP allowlist and MFA.
status.example.com	Public status page	Static or external status provider.

5.3 Rate Limiting Tiers

Tier	Map requests	Forecast API	AI briefing	Tracking
free	moderate; cache-heavy	limited hourly/site calls	not available or daily quota	basic only
pro	higher map and site quota	10-day forecast endpoints	limited briefings	standard live tracking
pro_plus	high-resolution premium layers	model comparison and XC endpoints	higher quota	enhanced tracking
club/school/event	pooled org quota	dashboard and bulk endpoints	briefing generation	multi-pilot tracking
rescue	priority operational quota	incident-specific access	allowed for incident context	emergency tracking export

6. Storage and State Architecture

The storage architecture uses different systems for different access patterns. PostgreSQL/PostGIS stores strongly relational and spatial data. TimescaleDB stores high-volume time-series forecasts, observations and telemetry. Redis handles low-latency cache, locks and short-lived queue state. Object storage handles large immutable artifacts.

6.1 Storage Matrix

Data type	Primary store	Retention	Notes
users, profiles, entitlements	PostgreSQL	account lifetime + legal retention	PII controlled by security volume.
launches, landings, obstacles	PostgreSQL/PostGIS	indefinite with version history	Spatial indexes required.
site forecasts and observations	TimescaleDB	hot 90-180 days; summarized long term	Hypertables and retention policies.
raw GRIB/NetCDF/Zarr	Object storage	configurable, normally 30-365 days	Keep enough for replay/debug.
processed map tiles/weather frames	Object storage + CDN	regenerate when stale	Versioned by model_run_id and layer.

live tracking positions	TimescaleDB + Redis hot cache	short hot window; user-controlled history	Safety and privacy sensitive.
ML models and feature artifacts	Object storage + registry DB	model lifecycle retention	Immutable model artifacts.
logs	Loki/object storage	30-180 days depending tier	No secrets or raw payment data.

6.2 PostgreSQL Deployment Options

Option	Use when	Pros	Risks
Self-hosted PostgreSQL on dedicated VM	MVP budget constrained and team can operate backups	Low cost, full control	Operational burden, failover complexity.
PostgreSQL StatefulSet in cluster	Dev/staging or small controlled deployments	Same Kubernetes operations model	Persistent volume and backup discipline required.
Managed PostgreSQL / Timescale service	Production with revenue and small team	Reduced operational risk	Higher cost, provider dependency.

6.3 Backup Classes

Backup class	Scope	Frequency	Restore target
database full backup	PostgreSQL and Timescale	daily	restore to staging monthly; production emergency RTO target.
WAL/archive backup	Postgres point-in-time recovery	continuous	point-in-time recovery.
object storage snapshot	raw weather, tiles, model artifacts	daily or bucket versioning	restore critical prefixes.
configuration backup	Helm values, Terraform state, secrets metadata	on change	recreate cluster from Git.
disaster export	minimal launch/site/alert operational snapshot	daily	read-only emergency mode.

7. Weather Pipeline Operations

Weather processing is the most operationally complex part of the platform. It must be queue-driven, replayable and transparent. Every model run must have a lifecycle record: discovered, downloading, downloaded, decoded, normalized, derived, scored, tiled, published, failed or superseded. The user-facing app must not display a model run until the publish gate validates core variables and site coverage.

7.1 Pipeline Stages

- Discover model run availability from each provider.
- Create weather_model_runs row with expected forecast hours and variables.
- Download raw artifacts into object storage with checksum metadata.
- Decode GRIB/NetCDF/Zarr into canonical variables and units.
- Create forecast cube partitions or chunked object artifacts.
- Generate site forecasts for launch database.
- Run terrain and paragliding-derived feature processors.
- Run validation: missing variables, unrealistic values, coverage gaps and model age.
- Generate map tiles/weather animation frames.
- Publish only after quality gate passes; otherwise mark stale or partial.

7.2 Queue Design

Queue	Producer	Consumer	Retry policy
weather.run.discovered	scheduler	weather-ingestion	retry provider checks with exponential backoff.
weather.file.downloaded	downloader	decoder	retry decode only if checksum valid.
weather.cube.ready	decoder	forecast-processor	retry chunk independently.
weather.site_forecast.ready	forecast-processor	risk-engine	retry per site/time window.
weather.tiles.requested	processor	tile-generator	deduplicate by layer/model_run_id.
alert.evaluate	forecast-publisher	alert-engine	priority queue for safety alerts.

7.3 Publish Gate

```
def can_publish_model_run(run):
    required = ['wind_speed', 'wind_direction', 'gust', 'temperature',
               'cloud_cover', 'precipitation_probability']
    if run.coverage_percent < 0.98:
        return False, 'coverage below threshold'
    if any(var not in run.variables for var in required):
        return False, 'missing required variable'
    if run.max_abs_wind_ms > 90 or run.min_temperature_c < -80:
        return False, 'unrealistic values detected'
    if run.site_forecast_count < run.expected_site_count * 0.95:
        return False, 'site forecast incomplete'
    return True, 'publishable'
```

8. AI/ML Operations

AI and ML workloads must be deployed with stronger governance than ordinary APIs because they influence safety recommendations. A model is not production-ready only because it produces good offline scores. It must be traceable, replayable, monitored and bounded by deterministic safety rules.

8.1 ML Runtime Services

Service	Responsibility	Runtime
feature-builder	Build online/offline features from forecasts, terrain, observations and site history	Batch workers + scheduled jobs
model-registry-adapter	Register models, model cards, promotion status and artifact URI	API + database
forecast-correction-service	Correct wind/gust/cloudbase/thermal variables per site	API + batch inference
risk-inference-service	Score turbulence, foehn, rotor, valley wind and storm risk	API + batch inference
ai-context-builder	Build grounded context for LLM briefings	API service
llm-gateway	Provider abstraction, prompt templates, safety filters, logging	API service

8.2 Model Promotion Gates

- Offline metrics improve over baseline on validation windows and do not degrade safety blockers.
- Backtest period excludes future leakage and uses only data available at forecast issue time.
- Model card is complete: data sources, feature set, limitations, failure modes and owner.
- Staging shadow mode runs for a minimum period and compares predictions with current model.
- Rollback model exists and can be activated without redeploying code.
- Monitoring dashboard includes drift, error by site, error by altitude and confidence calibration.

8.3 GPU Strategy

The MVP should not assume expensive GPUs. Most forecast correction, risk models and route ranking can begin with CPU inference using gradient boosting, linear models or small neural networks. GPU capacity becomes relevant for webcam computer vision, high-volume route simulation, self-hosted LLM inference or large-scale geospatial ML training. GPU workloads must be isolated from core APIs and should run as batch or async jobs unless low-latency inference is proven necessary.

9. CI/CD and Release Management

The release system must make deployments reproducible. Application code, infrastructure definitions, Helm charts, database migrations and OpenAPI schemas must be version-controlled. Production deployments should be automated but gated by checks because weather and safety systems need deterministic rollback.

9.1 Pipeline Stages

```
Pull Request:
  lint -> unit tests -> type checks -> OpenAPI diff -> Docker build -> security scan
```

Merge to main:
 integration tests -> migration dry-run -> push images -> deploy dev
 Release candidate:
 deploy staging -> smoke tests -> weather replay test -> mobile API contract tests
 Production:
 manual approval -> database backup check -> deploy canary -> monitor -> full rollout
 Rollback:
 helm rollback + disable feature flag + restore previous model version if needed

9.2 Required Checks

Check	Applies to	Failure response
OpenAPI compatibility	API services	Block release unless versioned breaking change is approved.
Migration dry-run	Database migrations	Block deployment; require migration fix.
Weather replay fixture	Weather engine	Block if derived values drift unexpectedly.
AI grounding regression	AI briefing	Block if hallucinated or unsupported advice appears.
Mobile contract tests	Public API	Block if required mobile app fields disappear.
Container vulnerability scan	All images	Block critical exploitable issues or document exception.

9.3 Branch and Versioning Strategy

- main: always deployable to dev.
- release/x.y: stabilization branch for production release candidates.
- hotfix/x.y.z: urgent production fixes with reduced scope and mandatory postmortem.
- Semantic versioning for services; API versioning via /v1, /v2 and OpenAPI schema IDs.
- Database migrations are forward-only by default; destructive migrations require two-phase rollout.

10. Secrets and Configuration

Secrets must never be stored in Git, Docker images, client apps or logs. Configuration is divided into public runtime config, environment-specific non-secret settings and secrets. Mobile apps must only receive public keys and public endpoint configuration.

Config class	Examples	Storage
public config	API base URL, map style ID, feature flags visible to app	versioned config endpoint / app bundle
environment config	database hostnames, queue names, object bucket names	ConfigMaps / Helm values
secrets	JWT signing keys, DB passwords, provider API tokens, Stripe secrets	Kubernetes Secrets or external secret manager
runtime flags	disable AI briefings, force stale weather banner, hide paid feature	feature flag service / admin DB
emergency config	maintenance mode, rescue-only mode, provider failover	admin-controlled signed config

10.1 Secret Rotation

22. Every external provider token has an owner, purpose and rotation schedule.
23. JWT signing keys support key IDs and overlapping rotation.
24. Database credentials are role-scoped and rotated separately for apps, migrations and backups.
25. Compromised mobile API keys are treated as public and must not authorize privileged actions.
26. Billing secrets are isolated from general app namespaces.

11. Observability

Observability is mandatory because the app combines safety-critical weather recommendations with paid services and live tracking. The platform needs metrics, logs, traces, synthetic checks, data quality dashboards and business telemetry. Prometheus-style metrics, Grafana dashboards and Loki-style logs are a practical open-source foundation.

11.1 Golden Signals

Area	Metrics
API	request rate, p95/p99 latency, error rate, saturation, rate-limit rejects.
Weather pipeline	model-run age, import duration, missing variables, publish success, stale site count.
Forecast quality	MAE by site, wind direction error, cloudbase error, model disagreement, confidence calibration.
Tracking	active WebSockets, position ingest latency, dropped messages, emergency share events.
Alerts	rules evaluated, notifications sent, delivery latency, failed push/email/SMS.
AI/ML	inference latency, fallback rate, blocked briefings, unsupported claims, drift score.
Billing	subscription webhooks, entitlement sync failures, payment event lag.

11.2 Log Fields

Standard structured log fields:

```
timestamp, level, service, environment, version, request_id, user_id_hash,  
org_id_hash, model_run_id, site_id, forecast_time, tier, latency_ms,  
error_code, retry_count, provider, queue_name, job_id
```

Prohibited log fields:

```
raw password, JWT, payment card data, private API token, full precise user location  
except in explicitly encrypted incident evidence storage.
```

11.3 Alert Rules

Alert	Condition	Severity
Weather data stale	primary model run older than threshold for target region	critical
Tracking latency high	p95 ingest-to-broadcast latency exceeds target	critical
DB replication lag	replica lag exceeds threshold	warning/critical
Alert delivery failure	push/email/SMS failures spike	critical for safety alerts
AI unsupported claim rate	safety filter blocks exceed threshold	warning
Tile cache miss storm	cache miss ratio spikes after deployment	warning
Object storage errors	5xx or timeout rate exceeds threshold	critical for weather pipeline

12. Backup and Disaster Recovery

Disaster recovery must prioritize safety and data integrity over feature completeness. If the full platform is degraded, the app should still be able to display stale banners, cached launch data, user alerts status and emergency contact/export functions. The system needs a documented RTO/RPO target per subsystem.

Subsystem	RTO target	RPO target	Recovery mode
public API	1 hour for MVP, 15 minutes mature	15 minutes	restore API deployment and DB connectivity.
weather forecasts	2 hours	last completed model run	serve last valid run with stale warning.
live tracking	15 minutes	5 minutes	degraded mode with direct emergency share.
billing	24 hours	provider source of truth	disable upgrades if uncertain; do not remove existing access.
AI briefing	24 hours	not safety-critical	fallback to deterministic summary.
launch database	4 hours	daily	read-only static cache if DB unavailable.

12.1 Restore Drill

27. Create isolated restore environment.
28. Restore latest database full backup and WAL to selected timestamp.
29. Restore critical object storage prefixes.

30. Run data integrity checks: users, launches, latest model runs, forecasts, alerts.
31. Run smoke tests against restored environment.
32. Record duration and failures; update RTO/RPO dashboard.

12.2 Emergency Read-Only Mode

Emergency read-only mode disables writes except operational incident logs. It serves cached map layers, latest valid site forecast, status notices and emergency contact functions. It must not generate fresh GO recommendations from incomplete weather data.

13. Security Operations Handoff

Full security design is covered in Volume 09, but infrastructure must already enforce baseline controls: least privilege, private networking, TLS everywhere, image scanning, audit logs, network policies and segregation of production secrets. The DevOps architecture must make secure operation the default path.

Control	Infrastructure implementation
TLS	Managed certificates at ingress; internal mTLS optional for mature phase.
Network isolation	Private DB network, Kubernetes NetworkPolicies, no public database ports.
Admin access	VPN or zero-trust access, MFA, audited sessions.
Container security	Non-root containers, read-only root FS where possible, minimal base images.
Secrets	External secrets or sealed secrets; no secrets in logs or images.
Audit	Admin and production changes logged with actor, timestamp and request ID.
Supply chain	Pinned image tags/digests, dependency scanning, SBOM generation.

14. Performance and Scaling

Scaling strategy must be driven by workload class. Map browsing is cache-heavy, weather processing is CPU and IO-heavy, tracking is latency-sensitive, and AI is quota/cost-sensitive. The system should avoid scaling everything together.

14.1 Scaling Signals

Component	Primary scaling metric	Secondary metric
public-api	p95 latency and CPU	request rate by tier
map-api	cache miss rate and response latency	tile generation backlog
forecast-processor	queue length	CPU/memory per chunk
tracking-gateway	active connections	message latency
alert-engine	alert evaluation lag	notification failure rate
ai-briefing	inference queue latency	provider token/cost quota
postgres	CPU, IOPS, locks, slow queries	connection pool saturation
redis	memory, evictions	command latency

14.2 Cache Strategy

- CDN caches static web assets, public tiles and public weather frames with versioned URLs.
- Redis caches site forecasts, launch rankings, entitlement checks and recent observations.
- No cache for payment mutation endpoints or privacy-sensitive tracking history.
- Cache keys include tier, region, model_run_id and language when outputs differ.
- Stale-while-revalidate is acceptable for map visuals but not for safety blocker decisions.

15. Cost Control

Cost control is a product requirement, not only an operations concern. The most expensive parts will likely be raw weather storage/processing, map tile generation, AI/ML inference, high-volume tracking and notification delivery. Paid tiers must map to measurable infrastructure cost drivers.

Cost driver	Control mechanism
weather imports	limit variables and regions for MVP; archive raw files; deduplicate model runs.
map tile generation	precompute only popular regions/layers; generate long-tail tiles on demand.
AI briefings	quota by tier; cache deterministic summaries; use smaller models for routine briefings.
tracking	sample adaptively; reduce frequency when grounded or stationary; user-controlled history retention.
object storage	lifecycle policies for raw weather, tiles, logs and model artifacts.
database	partitioning, retention, materialized summaries and query budgets.
notifications	push-first, email/SMS reserved for configured critical alerts.

15.1 Budget Tiers

Stage	Infrastructure posture	Goal
prototype	single VPS + Docker Compose + external weather APIs	prove product UX and scoring logic.
MVP beta	small K3s cluster + managed backups + object storage	serve initial pilot community reliably.
commercial V1	multi-node Kubernetes + observability + staged releases	support paid tiers and clubs.
scale V2	regional processing, CDN, ML pipelines, HA databases	global expansion and enterprise features.

16. Runbooks

Runbooks must be short, operational and executable under stress. Each runbook should include symptoms, impact, first checks, mitigation, rollback and post-incident tasks.

16.1 Weather Import Failure

33. Check provider status and latest weather_model_runs state.
34. Verify object storage writes and checksum failures.
35. Pause dependent tile generation if raw files are incomplete.
36. Publish stale-data banner for affected regions if model age exceeds threshold.
37. Trigger fallback provider import if configured.
38. Create incident note with provider, model_run_id and affected regions.

16.2 Tracking Degradation

39. Check active WebSocket count, CPU/memory and Redis latency.
40. Scale tracking-gateway replicas if saturation is high.
41. Disable non-essential telemetry enrichment.
42. Preserve emergency position sharing and last-known-position endpoints.
43. Notify users via status page if live map is degraded.

16.3 Bad Forecast Publish

44. Unpublish affected model_run_id and mark it invalid.
45. Revert active site forecasts to previous valid run.
46. Invalidate CDN/Redis keys for affected layers.
47. Disable AI briefings for affected region until context cache is rebuilt.
48. Run postmortem: data source, decoder, validation rule or derived feature failure.

17. MVP Infrastructure Slice

The MVP infrastructure should be deliberately smaller than the final architecture but must preserve the same interfaces. The MVP must prove user value, forecast ingestion, flyability scoring, alerting and mobile UX without overbuilding every enterprise feature.

MVP component	Implementation
---------------	----------------

Runtime	Docker Compose for dev; single K3s cluster for production beta.
Database	PostgreSQL with PostGIS and TimescaleDB extension, daily backup.
Cache/queue	Redis; Celery or worker queue.
Object storage	MinIO or S3-compatible bucket for raw weather, tiles and model artifacts.
Weather sources	Open-Meteo and MET Norway first; raw ECMWF/GFS pipeline as V1 expansion.
Map	MapLibre web/mobile with cached vector basemap and weather overlays.
Observability	Prometheus metrics, Grafana dashboards, centralized logs.
CI/CD	GitHub Actions build/test/deploy to staging; manual prod approval.
Security	TLS, JWT, RBAC baseline, secret separation, no public DB.

17.1 MVP Deployment Diagram

```

K3s cluster:
  namespace pip-prod
    ingress-nginx
    public-api x2
    map-api x2
    flyability-api x2
    weather-ingestion-worker x1
    forecast-processor-worker x1
    alert-engine x1
    tracking-api x1
    redis x1
outside or dedicated node:
  postgres-postgis-timescale
  minio/object-storage
observability:
  prometheus, grafana, loki/promtail

```

18. Roadmap and Backlog

Phase	Infrastructure work
MVP	K3s, PostGIS/Timescale, Redis, object storage, basic CI/CD, monitoring, backup, Open-Meteo/MET Norway ingestion.
V1	Raw ECMWF/GFS ingestion, vector tile pipeline, staged environments, canary releases, better alert routing.
V2	Regional processing, ML registry, feature store, shadow deployments, high availability database posture.
V3	Global CDN, multi-region read replicas, rescue/event priority infrastructure, advanced cost accounting.
V4	Enterprise compliance, dedicated customer environments, hardware weather station fleet operations.

18.1 Engineering Backlog

- Create Terraform/OpenTofu module for base network, cluster nodes, firewalls, load balancer and object storage.
- Create Helm chart per service plus shared library chart for probes, resources, labels and secrets.
- Create database migration pipeline with dry-run and production lock protection.
- Create weather pipeline replay fixture and regression dashboard.
- Create alert runbooks and synthetic checks for weather freshness, tracking and billing webhooks.
- Create cost dashboard by service, tier and pipeline stage.
- Create restore drill automation and monthly evidence report.

19. Appendices and Reference Manifests

19.1 Docker Compose Development Stack

services:

```

postgres:
  image: postgis/postgis:16-3.4
  environment:
    POSTGRES_DB: pip
    POSTGRES_USER: pip
    POSTGRES_PASSWORD: pip_dev_only
  ports: ["5432:5432"]
  volumes: ["pgdata:/var/lib/postgresql/data"]
redis:
  image: redis:7
  ports: ["6379:6379"]
minio:
  image: minio/minio
  command: server /data --console-address ':9001'
  ports: ["9000:9000", "9001:9001"]
public-api:
  build: ./services/public-api
  environment:
    DATABASE_URL: postgresql://pip:pip_dev_only@postgres:5432/pip
    REDIS_URL: redis://redis:6379/0
  depends_on: [postgres, redis]
volumes:
  pgdata: {}

```

19.2 Helm Values Skeleton

```

image:
  repository: registry.example.com/pip/public-api
  tag: 1.0.0
replicaCount: 3
resources:
  requests:
    cpu: 250m
    memory: 512Mi
  limits:
    cpu: 1000m
    memory: 1Gi
ingress:
  enabled: true
  host: api.example.com
config:
  ENVIRONMENT: production
  LOG_LEVEL: info
secrets:
  existingSecret: pip-runtime-secrets

```

19.3 Reference Sources

This document uses official/primary documentation for operational assumptions: Kubernetes documentation for container orchestration concepts; Hetzner documentation for cloud product and infrastructure references; Prometheus documentation for monitoring and alerting; Grafana Loki documentation for log aggregation architecture. Exact provider pricing and managed-service availability must be verified before procurement because they change over time.

End of Volume 08. Next volume: Volume 09 - Security Architecture v2 Technical.